CHƯƠNG 3 · BÀI 1

3.1. Try-catch & Custom Exception

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu ngoại lệ (exception) và phân loại ngoại lệ
> - Sử dụng try-catch-finally, throw và throws
> - Viết ngoại lệ tự định nghĩa

1. Ngoại lệ là gì

Ngoại lệ (exception) là sự kiện bất thường xảy ra khi chương trình đang chạy, làm gián đoạn luồng thực thi. Checked exception bắt buộc phải xử lý hoặc khai báo `throws`, ví dụ `IOException`. Unchecked exception kế thừa từ `RuntimeException`, ví dụ `NullPointerException` và `ArithmeticException`.

2. try-catch-finally

Khối `try` chứa đoạn mã có thể phát sinh ngoại lệ, khối `catch` bắt và xử lý ngoại lệ đó; một khối `try` có thể đi kèm nhiều khối `catch` cho các loại ngoại lệ khác nhau. Khối `finally` dùng để chứa đoạn mã luôn được thực thi dù có ngoại lệ hay không, thường để đóng file hoặc đóng kết nối.

```
try {
    int kq = 10 / soChia;
    System.out.println(kq);
} catch (ArithmeticException e) {
    System.out.println("Không chia được cho 0");
} finally {
    System.out.println("Kết thúc");
}
```

3. throw và throws

Từ khoá `throw` dùng để ném một ngoại lệ, ví dụ `throw new IllegalArgumentException("Sai giá trị")`. Từ khoá `throws` dùng để khai báo các ngoại lệ mà phương thức có thể ném ra ở chữ ký phương thức.

4. Ngoại lệ tự định nghĩa

Custom exception là lớp ngoại lệ do lập trình viên tự định nghĩa để mô tả lỗi nghiệp vụ, thường kế thừa từ `Exception` hoặc `RuntimeException`.

```
class InvalidPriceException extends RuntimeException {
    public InvalidPriceException(String msg) { super(msg); }
}

void datGia(double gia) {
    if (gia < 0) throw new InvalidPriceException("Giá không được âm");
}
```

Tóm tắt

- Checked phải xử lý hoặc `throws`; unchecked kế thừa `RuntimeException`.
- `finally` luôn chạy, dùng để giải phóng tài nguyên.
- `throw` ném ngoại lệ, `throws` khai báo ngoại lệ; có thể tự định nghĩa ngoại lệ nghiệp vụ.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết chương trình nhập hai số và xử lý ngoại lệ chia cho 0 bằng try-catch.
2. Viết ngoại lệ tự định nghĩa TuoiKhongHopLeException và ném ra khi tuổi âm.
3. Giải thích vì sao nên dùng `finally` khi làm việc với file.

Try-catch & Custom Exception — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: lỗi bị bỏ qua bằng catch rỗng khiến chương trình vẫn báo lưu thành công. Chỉ bắt lỗi tại nơi có thể xử lý, bổ sung ngữ cảnh và giữ nguyên nguyên nhân khi chuyển tiếp.

```
try {
    int quantity = Integer.parseInt("abc");
    System.out.println(quantity);
} catch (NumberFormatException error) {
    System.out.println("Số lượng phải là số nguyên");
}
```

Tự luyện: quy định lỗi nhập dữ liệu khác lỗi đọc file. **Hướng giải:** người dùng có thể sửa input, còn lỗi I/O cần thông báo phù hợp; không lộ thông tin nhạy cảm trong thông báo. Dùng try-with-resources cho tài nguyên AutoCloseable, không dựa vào việc đóng thủ công ở cuối đường chạy thành công.

Nguồn học tiếp

https://dev.java/learn/